

Toward Open-World Network Intrusion Detection via Open Recognition and Inspection

Lei Du¹, Yuhan Chai², Yan Jia, Binxing Fang³, *Member, IEEE*, Hao Li⁴, and Zhaoquan Gu⁵, *Member, IEEE*

Abstract—Deep learning is promising in open-world network intrusion detection, but current deep learning-based methods mainly focus on open recognition with properties that may not always hold and significantly neglect the inspection of unknown samples, increasing open space risks and manual inspection overhead for deployed models. To address these challenges in real-world environments, we propose a novel system, ORI, designed to tackle two critical tasks: 1) open recognition, including classifying known class samples while recognizing unknown ones, and 2) inspection, involving further inspecting samples recognized as unknown. Specifically, we reformulate open recognition as a binary classification task and propose a density-based method to recognize low-density samples as unknown while classifying known class samples with a closed-world classifier, thereby minimizing the risk associated with open spaces. To reduce the inspection overhead of samples recognized as unknown, we treat unknown sample inspection as a constrained clustering task, using a few manually inspected samples as constraints, and then assign labels to the remaining unknown samples via clustering. We evaluate our system against established open recognition and unknown sample inspection baselines through extensive experiments on three public datasets. Additionally, we simulated a security analyst inspecting unknown samples labeled by ORI. The experimental results demonstrate that ORI accurately classifies known class samples, recognizes unknown samples, and effectively labels samples recognized as unknown, enhancing both open recognition and inspection capabilities.

Index Terms—Network intrusion detection, open world network intrusion, open recognition, unknown sample inspection.

Received 16 December 2024; revised 14 July 2025; accepted 1 September 2025. Date of publication 10 September 2025; date of current version 23 September 2025. This work was supported in part by Shenzhen Science and Technology Program under Grant KJZD20231023094701003, in part by the Major Key Project of Pengcheng Laboratory under Grant PCL2024A05, in part by the National Natural Science Foundation of China under Grant 62372137, in part by the Tertiary Education Scientific Research Project of Guangzhou Municipal Education Bureau under Grant 2024312279, and in part by the Opening Project of Science and Technology on Communication Networks Laboratory under Grant FFX24641X007. The associate editor coordinating the review of this article and approving it for publication was Dr. Ashok Kumar Das. (Corresponding author: Zhaoquan Gu.)

Lei Du, Binxing Fang, and Zhaoquan Gu are with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Guangdong 518055, China, and also with the Department of New Networks, Pengcheng Laboratory, Shenzhen, Guangdong 518055, China (e-mail: 21b951040@stu.hit.edu.cn; fangbx@cae.cn; guzhaoquan@hit.edu.cn).

Yuhan Chai is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: chaiyuhuan@gzhu.edu.cn).

Yan Jia is with the College of Computer Science, National University of Defense Technology, Changsha 410073, China (e-mail: jiaayan2020@hit.edu.cn).

Hao Li is with Kunlun Digital Technology Company Ltd., Beijing 100010, China (e-mail: cuclihao@cuc.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3608666

I. INTRODUCTION

DEEP learning has been widely used in securing computer networks and endpoint devices against cyber attacks, including network intrusion detection [10], [17], [50], malware detection [14], [24], [35], log anomaly detection [9], [37], [51], and advanced persistent threat detection [1], [25], [46]. As a critical component of cybersecurity, network intrusion detection plays a vital role in analyzing network traffic behavior to identify potential intrusion attempts and provide early warnings. Heretofore, researchers have applied deep learning-based models for network intrusion detection systems and consistently achieved satisfactory performance [20]. Unfortunately, the superior performance of learning-based deep models typically works under the closed-world assumption that all types of attacks expected at inference would appear in training [40]. This assumption is inherently vulnerable and often violated in an open-world environment as the real network environment changes dynamically over time. Such changes include the natural evolution of benign applications, mutations in known attack vectors, the proliferation of new benign applications, and the emergence of previously unknown vulnerabilities [36]. As a result, deep models may confidently misclassify any unknown sample (whether benign or attack behaviors) encountered in the future into the known classes, which causes serious failures to the deployed models [12].

Recently, several works have been proposed to tackle network intrusion detection in an open-world environment, which can generally be divided into two solutions. The first focuses on recognizing or rejecting unknown samples without further analysis of the samples recognized as unknown [5], [47], [49], while the second concentrates on updating the deep model through periodic retraining or incremental learning with labeled unknown samples [19], [21], [55]. These solutions utilize multiple properties to recognize unknown samples, such as the reconstruction error of sample features [47] or the distance between samples and known class centroids [49]. However, these properties may not always hold and significantly increase the risk of open spaces. Moreover, most works focus on updating deep models rather than inspecting unknown samples, resulting in substantial labeling overhead when preparing samples for updates [6], [8]. Only previous work has attempted to inspect unknown samples by clustering the samples with the estimated number of classes [55]. As illustrated in Fig. 1, in the open-world environment, real-time captured samples recognized as unknown require further inspection at security operations centers. Nevertheless, a significant error in estimating the number of classes undermines

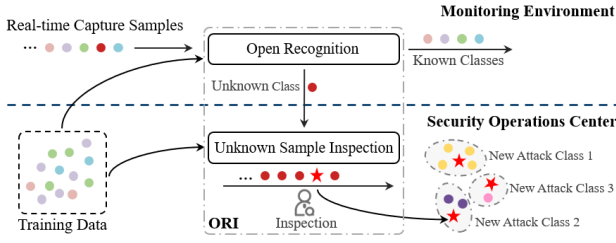


Fig. 1. Open recognition and inspection of samples.

the effectiveness of this method. Hence, the urgent priority for advancing open-world network intrusion detection is to develop a novel solution that recognizes unknown samples and inspects samples recognized as unknown.

A. Challenges

In short, tackling open recognition¹ and unknown sample inspection in open-world network intrusion detection is non-trivial and faces three major challenges. (1) How to learn generalizable feature representations: Deep models are typically trained on known class data, which can lead to overfitting known classes and poor generalization to encountered unknown samples [13]. (2) How to minimize open space risk: In the inference stage, test samples can appear anywhere in latent feature space, which raises the probability of misrecognition by deep models and increases the risk associated with open space [16], [41]. (3) How to efficiently inspect unknown samples: Inspecting unknown samples case by case is labor-intensive and time-consuming, and with the continuous emergence of unknown samples, promptly labeling all samples becomes impractical [15].

B. Our Work

In light of the mentioned challenges, we develop an open-world network intrusion detection system, ORI (short for **O**pen **R**ecognition and **I**nspection), for known class classification, unknown sample recognition, and further inspection and labeling of samples recognized as unknown. (1) To address the first challenge, we train a representation network with supervised contrastive learning to map samples of the same class into a compact feature space that supports the system construction by non-parametric methods. (2) To address the second challenge, we formulate unknown sample recognition as a binary classification task, where samples with low density relative to the training data are recognized as unknown, which helps the system distinguish between known class data and unknown samples during testing, thereby enabling the classification of known class samples while reserving unknown samples for further inspection. (3) To address the last challenge, we treat the inspection of unknown samples as a constrained clustering task. By using a few inspected unknown samples as constraints, the system can assign labels to the remaining unknown samples through clustering. The design

aims to minimize the labeling overhead, thereby reducing the inspection effort for security companies.

We conduct extensive experiments on three publicly available attack detection datasets to evaluate the performance of ORI. We compare ORI with four open recognition baselines in two experimental settings: pseudo multi-class classification and multi-class classification, and four unknown sample inspection baselines across two scenarios: with and without known classes present in samples recognized as unknown. We also conduct ablation experiments on open recognition and unknown sample inspection of ORI. In addition, we present a case study simulating a security analyst inspecting the labeling results of unknown samples provided by ORI. Experimental results demonstrate that ORI outperforms currently related baselines in both open recognition and unknown sample inspection, underscoring its effectiveness as a promising solution.

In summary, the main contributions of this paper are:

- We propose an open-world network intrusion detection system that can classify known class samples, recognize unknown samples, and inspect samples recognized as unknown. (§III-A)
- We propose a novel density-based method for open-world network intrusion recognition, which recognizes unknown samples based on the distribution density of known class data in reliable regions. (§III-B)
- To the best of our knowledge, this is the first work to explore unknown sample inspection through constrained clustering with a few labeled samples, significantly reducing the manual labeling budget. (§III-C)

The remainder of this paper is organized as follows. In Section II, we outline the background of open-world network intrusion and define the problem scope of the work. Then, we present our system in detail in Section III, followed by a comprehensive evaluation in Section IV. We review related work in Section V and provide several discussions in Section VI. Finally, we conclude the paper in Section VII.

II. BACKGROUND AND PROBLEM SCOPE

In this section, we provide the background of deep learning-based network intrusion detection in an open-world environment and define the scope of our problem.

A. Open-World Network Intrusion Detection

Deep learning-based models for network intrusion detection in an open-world environment must effectively classify known class samples, recognize unknown samples, inspect samples recognized as unknown, and update the deep models with the recently inspected ones [4], [31], [55]. Although considerable efforts have been made in these areas, research has often been conducted independently, with only a few works integrating multiple capabilities. With extensive research and development, the classification of known class samples has significantly advanced [20]. However, most methods rely on a closed-world assumption [41].

1) *Threat Model*: In the context of network intrusion detection, a well-trained closed-world deep detection model may be vulnerable to unknown samples, which forces deep models to

¹This paper refers to known class classification and unknown sample recognition as open recognition.

misclassify unknown samples as one of the known classes, leading to significant failures in real-world deployments. Such unknown samples can emerge from three sources: unknown class samples not present during training, known class samples deviating from the expected distribution, and adversarial samples deliberately manipulated by the attacker [43]. We primarily focus on the first type, while the other types will be deferred to future work.

Recent work has explored open recognition in a pseudo multi-class classification setting, which utilizes classifier confidence [18], feature reconstruction errors [47], and the distances between feature vectors and class centroids [49], [55] to recognize unknown samples while classifying known samples. However, these methods have weaknesses since such properties are not foolproof. Other works have investigated updating deep models through periodic retraining or incremental learning with labeled unknown samples [8], [55], but it is impractical due to the resource-intensive and time-demanding nature of labeling [45]. Prior work [55] inspects unknown samples with minimal overhead through clustering, but significant errors in estimating the number of classes hinder its effectiveness. While incremental updating of deep models is a valuable research topic, it is beyond the scope of this work.

2) *Our Goal*: We aim to develop an open-world network intrusion detection system that tackles open recognition and unknown sample inspection toward more realistic scenarios. Our system is designed to meet the following requirements:

- **Support fine-grained class definitions**: By incorporating detailed attack type definitions for both normal and attack classes, our system enables administrators to configure more flexible security policies and enhance detection capabilities.
- **Efficient unknown sample inspection**: The system is intended to inspect only a limited number of unknown samples, reducing the overhead for security analysts.

B. Problem Formulation

Formally, let $\mathbb{D}_{kn} = \{(x_i, y_i) \mid y_i \in \mathcal{Y}_{kn}\}$ represent the training data collected from an internal network (e.g., an enterprise network), where each sample is a bi-flow or a session between two hosts. Here, y_i is the label of x_i , and $\mathcal{Y}_{kn}$ includes all known classes $\{N_1, \dots, N_n, A_1, \dots, A_a\}$, with normal classes $\{N_1, \dots, N_n\}$ and attack classes $\{A_1, \dots, A_a\}$. As the real network environment changes over time, test samples $\mathbb{D}_{un} = \{x_i \mid y_i \in \{\mathcal{Y}_{kn} \cup \mathcal{Y}_{un}\}\}$ are captured in real-time. These test samples may include known classes $\mathcal{Y}_{kn}$, unknown normal classes $\{N_{n+1}, \dots, N_{n+b}\} \subset \mathcal{Y}_{un}$, and unknown attack classes $\{A_{a+1}, \dots, A_{a+m}\} \subset \mathcal{Y}_{un}$. Suppose f_θ is a representation network trained with $\mathbb{D}_{kn}$, parameterized by θ , and the security company can afford to label M samples.

- **Open recognition**: The representation network f_θ is leveraged to build an open recognition module. This module recognizes test samples in $\mathbb{D}_{un}$ from $\mathcal{Y}_{un}$ as *unknown* (indicated by the masked True) while continuing to classify test samples from $\mathbb{D}_{un}$ that belong to $\mathcal{Y}_{kn}$ into $y \in \mathcal{Y}_{kn}$. The samples recognized as unknown are denoted by $\mathbb{U}$, where $|\mathbb{D}_{un}| \neq |\mathbb{U}|$.

- **Unknown sample inspection**: f_θ is also utilized to devise an unknown sample inspection module, where M samples from $\mathbb{U}$ are selected for manual inspection by security analysts, and then labels are assigned to the remaining unknown samples through clustering. The inspected samples $\widetilde{\mathbb{U}} = \{(x_i, y_i) \mid y_i \in \{\widetilde{\mathcal{Y}}_{kn} \cup \widetilde{\mathcal{Y}}_{un}\}\}$ include new class samples (x_i, y_i) , where $y_i \in \widetilde{\mathcal{Y}}_{un} \subset \mathcal{Y}_{un}$, and previously known class samples (x_j, y_j) , where $y_j \in \widetilde{\mathcal{Y}}_{kn} \subset \mathcal{Y}_{kn}$.

To enhance the representation of $\mathbb{U}$, the representation network f_θ can be trained from scratch with $\mathbb{D}_{kn}$ and $\mathbb{U}$, reparameterized by φ .

III. METHODOLOGY

In this section, we develop an open-world network intrusion detection system, ORI (short for **O**pen **R**ecognition and **I**nspection), to tackle open recognition and unknown sample inspection, which consists of two modules. To facilitate understanding, we provide an overview with a simplified example in §III-A to illustrate the workflow of ORI and present the details of each module in §III-B and §III-C.

A. Overview and Simplified Example

We design our system based on the observation that a representation network can serve as a non-parametric nearest neighbor classifier without imposing distributional assumptions about the underlying feature space. Many previous works in network intrusion detection leverage this characteristic when developing deep models [32], [49]. This insight inspired us to separate the learning of feature representations for network traffic from the development of decision processes for open recognition and unknown sample inspection.

As shown in Fig. 2, our system performs in an online fashion, continuously recognizing and inspecting samples from an internal network. These samples are either classified into known classes or recognized as unknown. In practice, the number of samples recognized as unknown may vary depending on the network environment and the intensity of cyber attacks. Hence, unknown samples are typically inspected periodically or when the number of samples reaches a predefined threshold. In this example, assume we have collected training data with known classes $\{N_1, N_2, A_1, A_2\}$. After some time, test samples $\{x_1^{N_1}, x_2^{N_2}, x_3^{N_2}, x_4^{A_1}, x_5^{A_2}, x_6^{N_3}, x_7^{N_4}, x_8^{N_4}, x_9^{N_4}, x_{10}^{A_3}, x_{11}^{A_3}, x_{12}^{A_3}, x_{13}^{A_3}, x_{14}^{A_4}, x_{15}^{A_4}\}$ are captured in real time from the internal network. Here, x_i^j denotes the i -th sample with the ground truth label j , and the labels are inaccessible.

The ORI procedure consists of four main steps. Initially, we train a representation network f_θ to extract feature vectors of the training data. In step ①, we leverage these feature vectors to construct a known class or closed-world classifier for classifying the test samples. Clearly, test samples from unknown classes are misclassified as known classes. For instance, an unknown sample $x_6^{N_3}$ is misclassified as $\widetilde{y}_6^{N_2}$. Then, in step ②, we propose a density-based method for recognizing samples that do not belong to the training data distribution. This method formulates a binary classification task, where test samples with low density in the feature space of the training data are masked as unknown. Intuitively, recognizing unknown samples at the density distribution level is more effective.

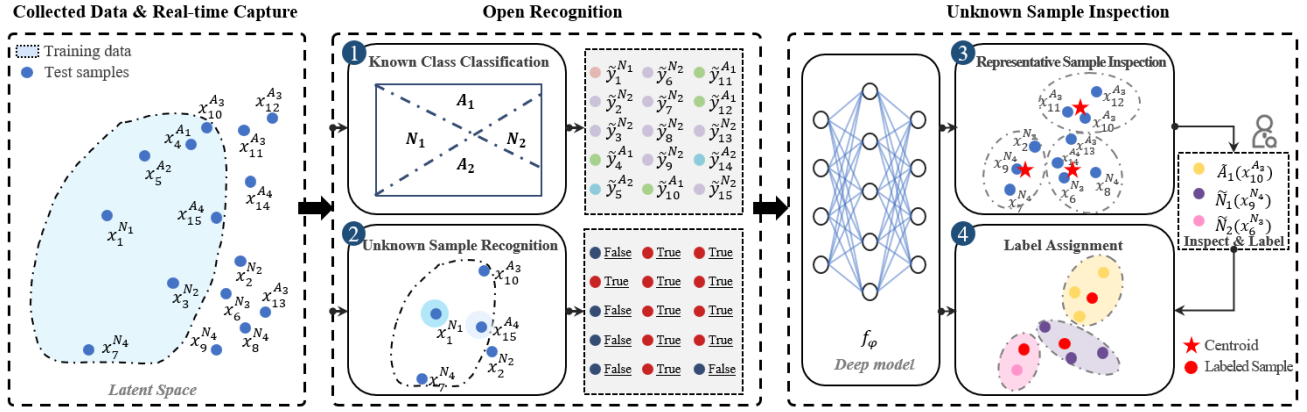


Fig. 2. Overview and illustration of ORI with an example.

Nevertheless, insufficient information in the training data may lead to the misrecognition of some test samples. As shown in Fig. 2, an unknown sample $x_{15}^{A_4}$ can be incorrectly recognized as a known class sample, while a known class sample $x_2^{N_2}$ can be wrongly marked as unknown. In step ③, we train a representation network f_ϕ to obtain more tailored feature vectors for samples recognized as unknown. We then select a few representative unknown samples for manual inspection by security analysts. Analysts inspect the selected samples case by case and identify new classes. For example, samples $x_6^{N_3}$, $x_9^{N_4}$ and $x_{10}^{A_3}$ are selected and inspected, and new classes A_1 , N_1 , and N_2 are recorded. Ideally, clustering samples into several clusters and selecting representatives from each cluster for labeling would accurately label unknown samples, but this is not the case. Therefore, in step ④, we propose a constrained clustering method for labeling uninspected unknown samples. This method used manually labeled samples and recorded new classes as constraints to label the remaining unknown samples through clustering. Finally, the ORI procedure concludes by combining the results of open recognition and unknown sample inspection to assess the current cybersecurity situation.

B. Open Recognition

The open recognition module in this work aims to classify known class samples and recognize unknown samples. The process of open recognition is shown in Algorithm 1.

1) *Representation Learning*: We utilize a representation network trained with supervised contrastive learning [28] to extract feature vectors of the input sample. This method is motivated by two key aspects. First, supervised contrastive learning has been demonstrated to effectively map data from the same class into a compact feature space. Second, the representation network can serve as a robust nearest neighbor classifier, which suggests that non-parametric clustering methods perform well within its feature space.

Formally, let x_i be a randomly selected sample from a mini-batch B_{kn} . We optimized f_θ over the training data $\mathbb{D}_{kn}$ by minimizing the supervised contrastive loss as follows:

$$\mathcal{L}(\theta) = - \sum_{i \in B_{kn}} \frac{1}{|\mathcal{N}_{kn}(i)|} \sum_{j \in \mathcal{N}_{kn}(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_n 1_{[n \neq i]} \exp(\mathbf{z}_i \cdot \mathbf{z}_n / \tau)} \quad (1)$$

Algorithm 1 Open Recognition

Input: Training data $\mathbb{D}_{kn}$, test samples $\mathbb{D}_{un}$, the pre-trained representation network f_θ , the number of local samples k_1 , and the number of reliable samples k_2 .

Output: Known class predictions $\tilde{\mathcal{Y}}$ and unknown samples $\mathcal{U}$.

▷ Collect feature vectors $\mathcal{Z}_{kn}$ of $\mathbb{D}_{kn}$ by f_θ .

- 1: **for** index $i = 1$ to $|\mathbb{D}_{kn}|$ **do**
- 2: $\mathcal{Z}_{kn}(\mathbf{z}_1, \dots, \mathbf{z}_i) \leftarrow f_\theta(x_i) / \|(f_\theta(x_i))\|_2$
- 3: **end for**
- ▷ Construct a classifier for known classes.
- 4: $\text{KNN} = \text{KNeighborsClassifier}(\mathcal{Z}_{kn}, \mathcal{Y}_{kn})$
- ▷ Construct a recognizer for unknown samples.
- 5: **for** index $i = 1$ to $|\mathcal{Z}_{kn}|$ **do**
- 6: $\mathcal{Z}_{k_1}(\mathbf{z}_i) \leftarrow \|\mathbf{z}_i - \mathbf{z}\|_2, \mathbf{z} \in \mathcal{Z}_{kn}$
- 7: $\rho(\mathbf{z}_i) \leftarrow \frac{1}{k_1} \sum_{j=1}^{k_1} d(\mathbf{z}_j, \mathbf{z}_i), \mathbf{z}_j \in \mathcal{Z}_{k_1}(\mathbf{z}_i)$
- 8: **end for**
- 9: $\delta \leftarrow \frac{1}{|\mathcal{Z}_{kn}|} \sum_{i=1}^{|\mathcal{Z}_{kn}|} \rho(\mathbf{z}_i)$
- 10: ▷ Classify test samples as known classes or unknown.
- 11: **for** index $i = 1$ to $|\mathbb{D}_{un}|$ **do**
- 12: $\mathbf{z}_i \leftarrow f_\theta(x_i) / \|(f_\theta(x_i))\|_2$
- 13: $\mathcal{Z}_{k_1}(\mathbf{z}_i) \leftarrow \|\mathbf{z}_i - \mathbf{z}\|_2, \mathbf{z} \in \mathcal{Z}_{kn}$
- 14:

$$\tilde{\mathcal{Y}}_i \leftarrow \begin{cases} \text{Unknown} & \text{if } \sum_{i=1}^{k_1} 1(d(\mathbf{z}_j, \mathbf{z}_i) \leq \delta) \leq k_2, \\ \text{Known} & \text{otherwise, classification by KNN.} \end{cases}$$

15: **end for**

Here, $B_{kn} \subset \mathbb{D}_{kn}$, and $\mathcal{N}_{kn}(i)$ are the indices of other samples with the same label as x_i in the mini-batch B_{kn} . The feature vector of x_i denoted by $\mathbf{z}_i$, is given by $\mathbf{z}_i = f_\theta(x_i) \in \mathbb{R}^m$, where f_θ is the representation network. The indicator function is represented by 1, and $\tau \in \mathbb{R}^+$ is a scalar temperature parameter.

2) *Known Class Classification*: Given the feature representations of the training data, we classify test samples into known classes using the k -nearest neighbors (KNN) algorithm with majority voting instead of performing on parametric methods. KNN is an instance-based learning method that directly stores the training data without a training stage. We extract feature

vectors for the train data using the representation network f_θ , denoted as $\mathbb{Z}_{kn} = (\mathbf{z}_1, \mathbf{z}_2, \dots)$, where each feature vector $\mathbf{z} = f_\theta(x) / \|f_\theta(x)\|_2$ is normalized. By leveraging the feature vectors $\mathbb{Z}_{kn}$ and the corresponding ground truth labels $\mathcal{Y}_{kn}$, we construct a classifier for known class classification.

In the inference stage, we derive the normalized feature vector $\mathbf{z}^*$ for a test sample x^* and calculate the Euclidean distance between $\mathbf{z}^*$ and the feature vectors in $\mathbb{Z}_{kn}$. By employing these distances, we identify the labels of the k nearest neighbors $\mathcal{Y}_k(x^*) = (y_1, \dots, y_k)$. The predicted label $\tilde{y}$ for the test sample x^* is then determined by majority voting:

$$\tilde{y} = \arg \max_y \sum_{j=1}^k 1[y_j = y] \quad (2)$$

Here, $y_j \in \mathcal{Y}_k(x^*)$ and $1[y_j = y]$ is an indicator function that returns 1 if $y_j = y$, and 0 otherwise. Note that while we recommend non-parametric methods in ORI, parametric methods are not excluded for classifying known classes.

3) *Unknown Sample Recognition*: Deep models trained on known class data often struggle to generalize to unknown samples. To tackle this issue, we formalize unknown sample recognition as a binary classification task, where samples from known classes are marked as False and unknown samples are True. Leveraging the compact feature space learned by the representation network, we propose a density-based method to recognize unknown samples. For each sample x^* in the training data, we index the corresponding feature vector $\mathbf{z}^*$, calculate the distances to the feature vectors in $\mathbb{Z}_{kn}$, and select the k_1 nearest neighbors $\mathbb{Z}_{k_1}(\mathbf{z}^*)$ to calculate the local density of the sample x^* :

$$\rho(\mathbf{z}^*) = \frac{1}{k_1} \sum_{i=1}^{k_1} d(\mathbf{z}, \mathbf{z}^*), \mathbf{z} \in \mathbb{Z}_{kn} \quad (3)$$

Here, $d(\mathbf{z}, \mathbf{z}^*)$ is the Euclidean distance between $\mathbf{z}$ and $\mathbf{z}^*$. We then calculate the global density based on the local densities of all training data, and establish a distance threshold δ to define the reliable region. This threshold δ ensures a high local density for known class data while maintaining a low density for unknown samples. Since $\mathbf{z}^*$ is a normalized feature vector in $\mathbb{R}^m$, the reliable region corresponds to the area on the m -dimensional unit hyper-sphere within a Euclidean distance of δ from the center $\mathbf{z}^*$.

In the inference stage, we first derive the normalized feature vector $\mathbf{z}^*$ for a test sample x^* , then identify the k_1 nearest neighbors $\mathbb{Z}_{k_1}(\mathbf{z}^*)$ by calculating the Euclidean distance between $\mathbf{z}^*$ and the feature vectors in $\mathbb{Z}_{kn}$. We generalize the reliable region to the test sample x^* by counting the number of training data within this region. If at least k_2 samples from the k_1 nearest neighbors lie within the distance threshold δ , the test sample x^* is recognized as known; otherwise, it is recognized as unknown:

$$\sum_{i=1}^{k_1} 1(d(\mathbf{z}_i, \mathbf{z}^*) \leq \delta) \leq k_2, \mathbf{z}_i \in \mathbb{Z}_{k_1}(\mathbf{z}^*) \quad (4)$$

where 1 is the indicator function that returns 1 if the specified condition is met and 0 otherwise, and k_2 is the number of reliable samples.

4) *Theoretical Analysis*: Assume the local density distributions for known class data (kn) and unknown samples (un) as follows:

$$\begin{aligned} \rho(\mathbf{z}^*)|y=0 &\sim N(\mu_{kn}, \sigma_{kn}^2) \\ \rho(\mathbf{z}^*)|y=1 &\sim N(\mu_{un}, \sigma_{un}^2) \end{aligned} \quad (5)$$

where μ and σ represent the mean and variance of the local density, respectively. By applying Bayes' theorem, the probability that $\mathbf{z}^*$ belongs to known classes is given by:

$$\hat{p}(y=0|\hat{\rho}(\mathbf{z}^*)) = \frac{\hat{p}_{kn}(\mathbf{z}^*) \cdot p(y=0)}{\hat{p}_{kn}(\mathbf{z}^*) \cdot p(y=0) + \hat{p}_{un}(\mathbf{z}^*) \cdot p(y=1)} \quad (6)$$

where $\hat{p}_{kn}(\mathbf{z}^*) = p(\hat{\rho}(\mathbf{z}^*) | y=0)$ and $\hat{p}_{un}(\mathbf{z}^*) = p(\hat{\rho}(\mathbf{z}^*) | y=1)$ are the probability distribution of the local density $\hat{\rho}(\mathbf{z}^*)$ for known class data and unknown samples, respectively. As k_1 increases, the local density estimate $\hat{\rho}(\mathbf{z}^*)$ converges to the true local density $\rho(\mathbf{z}^*)$:

$$\lim_{k_1 \rightarrow \infty} \hat{\rho}(\mathbf{z}^*) = \rho(\mathbf{z}^*) = \mathbb{E}[\hat{\rho}(\mathbf{z}^*)] \quad (7)$$

and the variance of the local density estimate can be approximated as:

$$\text{Var}[\hat{\rho}(\mathbf{z}^*)] = \frac{1}{k_1^2} \sum_{i=1}^{k_1} \text{Var}[d(\mathbf{z}_i, \mathbf{z}^*)] \quad (8)$$

Hence, selecting an appropriate k_1 can ensure that the variance of the local density estimate is slight enough to accurately and reliably reflect the actual global density. Moreover, choosing a suitable k_2 can guarantee sufficient samples in the known class data to satisfy the density condition, thereby minimizing the open space risk and enhancing the open recognition accuracy.

C. Unknown Sample Inspection

In the unknown sample inspection module, we further inspect samples recognized as unknown provided by the open recognition module while maintaining limited overhead for manual inspection.

1) *Relearning Representations*: The existing representation network f_θ , which is trained solely on known class data, inherently generates biased representations for unknown samples. To mitigate this issue and improve the representation of unknown samples, we introduce a new representation network f_φ . This representation network f_φ is jointly trained with supervised contrastive learning on the known class data and reconstruction learning for all samples. Formally, we optimized f_φ over both the known class data $\mathbb{D}_{kn}$ and the unknown samples $\mathbb{U}$ by minimizing the following objective function:

$$\begin{aligned} \mathcal{L}(\varphi) &= (1 - \lambda) \frac{1}{|B|} \sum_{j \in B} \|x_j - \tilde{x}_j\|_2^2 \\ &\quad + \lambda \sum_{i \in B_{kn}} - \frac{1}{|\mathcal{N}_{kn}(i)|} \sum_{j \in \mathcal{N}_{kn}(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_n 1_{[n \neq i]} \exp(\mathbf{z}_i \cdot \mathbf{z}_n / \tau)} \end{aligned} \quad (9)$$

where λ is a weight coefficient, and $B \subset (\mathbb{D}_{kn} \cup \mathbb{U})$. x_j is the original input sample, and $\tilde{x}_j$ is its reconstruction. The first term corresponds to the reconstruction loss, which minimizes

the mean squared error between x_j and $\tilde{x}_j$ for the samples in the mini-batch B . The second term is supervised contrastive loss, which incorporates prior knowledge into semantically meaningful representations, thereby helping to mitigate overfitting to known classes.

2) *Representative Sample Inspection*: Since unknown samples may continuously emerge, comprehensively and promptly inspecting all unknown samples is impractical in real-world environments. Therefore, we select a few representative samples from unknown samples for manual inspection and labeling by security analysts. These representative samples need to satisfy two criteria: First, the number of selected samples should be small ($M \ll |\mathbb{U}|$) to ensure that security companies can afford the overhead of manual inspection. Second, the selected samples should be diverse to cover all potential unknown classes, hence M should be at least equal to the number of ground truth classes.

To achieve this, we employ two selection strategies: random sampling and k -means sampling. In random sampling, we randomly selected M samples from the unknown samples. In k -means sampling, we first cluster the feature vectors of unknown samples using k -means, and the number of clusters is set to M . We select the sample closest to each cluster centroid based on the Euclidean distance between the feature vectors and centroids. Then, security analysts inspect the selected M samples, providing feedback with the ground truth labels $\mathcal{Y}_S$ of the selected unknown samples $\mathcal{S}$, as well as the estimated number of classes K .

3) *Label Assignment*: To avoid additional manual inspection overhead, we treat the labeling of the remaining unknown samples as a constrained clustering task. Specifically, the number of clusters is set to the estimated number of classes K , with the constraint that samples from the same class in the labeled samples $\mathcal{S} \times \mathcal{Y}_S$ are consistently grouped into the same cluster. The optimization objective of clustering under these constraints is expressed as:

$$\mathcal{J} = \sum_{j=1}^K \sum_{\mathbf{z}_i \in \mathcal{C}_j} \|\mathbf{z}_i - c_j\|_2^2$$

s.t. $\forall t, \exists j$ such that $\mathbf{z}_i \in \mathcal{C}_j, \forall \mathbf{z}_i$ with label t . (10)

where $\mathbf{z}_i$ denotes a sample in cluster $\mathcal{C}_j$, and c_j denotes the j -th centroid.

We implement this idea by modifying the k -means++ algorithm [2], as shown in Algorithm 2. Initially, the K centroids are calculated as the average of the feature vectors for each class in $\mathcal{Y}_S$, where c_i is the centroid of i -th cluster. (lines 8-10). The clustering process then assigns labels to the remaining unknown samples. We initialize the iteration counter $iter$ to 0 and the convergence flag $convergence$ to false (line 11). For each sample x^* in the remaining unknown samples $\mathbb{U} \setminus \mathcal{S}$, we calculate the Euclidean distances to the K centroids and assign the label with the nearest centroid (lines 13-16). Once all unknown samples are labeled, they form the set of labeled unknown samples $\tilde{\mathbb{U}}$ (line 17), and the flag $convergence$ is set to true (line 18). Then, the centroids are updated based on $\tilde{\mathbb{U}}$, with the constraint that samples from the same class in $\mathcal{S} \times \mathcal{Y}_S$ remain in the same cluster. If the centroids change, the flag $convergence$ is reset to false; otherwise, it remains unchanged

Algorithm 2 Unknown Sample Inspection

Input: Unknown samples $\mathbb{U}$, the pre-trained representation network f_φ , and the number of inspected samples M .

Output: The number of classes K , the labeled samples $\tilde{\mathbb{U}}$.

```

1: for index  $i = 1$  to  $|\mathbb{U}|$  do
2:    $\mathbb{Z}_{un}(\mathbf{z}_1, \dots, \mathbf{z}_i) \leftarrow f_\varphi(x_i) / \|f_\varphi(x_i)\|_2$ 
3: end for
4:
5:  $\mathcal{S}, \mathbb{Z}_S \leftarrow \text{sampleSelector}(\mathbb{U}, \mathbb{Z}_{un}, M)$ 
6:  $K, \mathcal{Y}_S \leftarrow \text{Inspection of } \mathcal{S} \text{ by security analysts.}$ 
7:
8:  $\triangleright$  Select representative samples.
9:  $\triangleright$  Assign labels for  $\mathbb{U} \setminus \mathcal{S}$ 
10: for each class  $i \in \{1, 2, \dots, K\}$  do
11:    $c_i \leftarrow \frac{\sum_{j \in \mathcal{S} \mid [i=y]} \mathbf{z}_j}{|\mathcal{S} \mid [i=y]|}, y \in \mathcal{Y}_S$   $\triangleright$  Initialize centroids.
12: end for
13: Initialize iteration counter  $iter \leftarrow 0$  and convergence flag  $convergence \leftarrow \text{false}$ .
14: while  $iter < T$  and  $\neg convergence$  do
15:   for each index  $j \in \mathbb{U} \setminus \mathcal{S}$  do
16:      $d_i^{(j)} \leftarrow \|\mathbf{z}_j - c_i\|_2, i = 1, \dots, K$ 
17:      $\mathcal{Y}_{un}(\tilde{y}_1, \tilde{y}_j, \dots) \leftarrow \arg \min_i d_i^{(j)}$ 
18:   end for
19:    $\tilde{\mathbb{U}} \leftarrow \{(x_j, y_j) \mid x_j \in \mathbb{U}, y_j \in \{\mathcal{Y}_S \cup \tilde{\mathcal{Y}}_{un}\}\}$ 
20:   Set  $convergence \leftarrow \text{true}$ 
21:   for each class  $i \in \{1, 2, \dots, K\}$  do
22:      $\triangleright$  Update centroids.
23:      $c_i \leftarrow \frac{1}{|\mathbb{Z}_{un} \mid [i=y]|} \sum_{j \in \mathbb{Z}_{un} \mid [i=y]} \mathbf{z}_j, y \in \{\mathcal{Y}_S \cup \tilde{\mathcal{Y}}_{un}\}$ 
24:     if centroids have changed then
25:        $convergence \leftarrow \text{false}$ 
26:     end if
27:   end for
28:    $iter \leftarrow iter + 1$ 
29: end while
```

(lines 19-25). This process repeats until either $iter$ reaches the maximum limit T or the centroids stabilize (lines 12-27).

IV. EVALUATION

In this section, we first present the experimental setup (§IV-A). We then evaluate the effectiveness of ORI with several baseline methods (§IV-B), followed by a detailed analysis of ORI, including ablation studies (§IV-C) and an examination of various factors (§IV-D). Finally, we provide a use case to simulate security analysts inspecting the unknown sample labeling results of ORI (§IV-E).

A. Experimental Setup

1) *Datasets*: We evaluate ORI using two public datasets: USTC-TFC2016 [44] and CSE-CIC-IDS2018 [38], both are widely used in network intrusion detection research. The USTC-TFC2016 dataset contains 20 types of network traffic, half of which represent normal activity and the other half represent activity generated by malware. Following [55], we extracted byte features from the first 784 bytes of each

TABLE I
CLASS STATISTICS OF THE DATASETS USED IN THIS WORK

Dataset		Normal	Malicious
USTC-TFC2016	Known	BitTorrent	Shifu
		Outlook	Cridex
		WorldOfWarcraft	Neris
		MySQL	Tinba
	Unknown	Gmail	Zeus
		Weibo	Nsis-ay
		FTP	Htbot
		Facetime	Miuref
CIC-IDS2018	Known		DoS Hulk
		FTP	DDoS-HOIC
		Weibo	DDoS-LOIC-HTTP
		MySQL	Botnet Ares
	Unknown	SMB	SSH-BruteForce
		Gmail	Infiltration-NMAP Portscan
		WorldOfWarcraft	DoS GoldenEye
		BitTorrent	DoS Slowloris
		Outlook	DDoS-LOIC-UDP
		Skype	
TON-IoT	Known	Benign	DDoS
			Scanning
			Injection
			Password
	Unknown	\	XSS
			Mitm
			Backdoor
			Ransomware
			DoS

flow. The original CSE-CIC-IDS2018 dataset captures various types of network traffic from hundreds of hosts in an internal network, including multiple malicious classes and one normal class. We extracted statistical features from the flows in this dataset by using the CICFlowMeter tool. Since CSE-CIC-IDS2018 only provides one normal class without fine-grained labels, we excluded the normal class and supplemented the dataset with normal traffic from USTC-TFC2016, which we refer to as CIC-IDS2018. Note that the version of the CICFlowMeter used in this work was modified as described in [11]. Moreover, we use the TON-IoT dataset [33], an Internet of Things network security dataset, to evaluate ORI. This dataset includes one normal class and nine attack classes, with samples from each class comprising 30 statistical features.

2) *Evaluation Protocols*: For open recognition, a deep model is trained solely on samples from known classes, while testing includes both known and unknown class samples. The class statistics used for evaluation are shown in TABLE I. We considered two evaluation settings: pseudo multi-class classification and multi-class classification. In the multi-class classification setting, fine-grained labels are also provided for the normal class. The known class data were randomly split into training and testing samples with an 8:2 ratio, and all unknown class samples were included in the testing samples. In the inference stage, the deep model classifies known class samples and recognizes unknown class samples in an open-world environment, where the latter are inspected with minimal manual overhead.

For unknown sample inspection, we evaluate two scenarios. In the first scenario, the unknown samples consist of both ground truth unknown classes and misclassified known classes, representing a more realistic and challenging setting. In both scenarios, security analysts inspect a subset of unknown samples to define constraints for assigning labels to the remaining unknown samples through clustering.

3) *Metrics*: We use multi-class classification accuracy to evaluate the open recognition performance, where samples from unknown classes are treated as a single unknown class. We also measure the known class accuracy in a closed-world environment, where unknown samples are excluded during testing. In addition, we calculate the Weighted Average F1-score (WAF), which weights the F1-score of each class according to its sample proportion relative to the total number of samples. Given the significant class imbalance between known class data and unknown samples, we also compute a variant of this metric, the Weighted Average F1-score for the unknown class (U-WAF).

For the inspection of unknown samples, we report the estimated number of unknown classes to provide security analysts with insights into the diversity and potential risk levels of these samples. Clustering accuracy is also evaluated by comparing the ground truth labels y_i with deep model predictions $\tilde{y}_i$ as follows:

$$\text{Cluster Accuracy} = \max_{p \in \mathcal{P}(\mathcal{Y}_{un})} \frac{1}{|\tilde{\mathbf{U}}|} \sum_{i=1}^{|\tilde{\mathbf{U}}|} 1\{y_i = p(\tilde{y}_i)\}$$

where $\mathcal{P}(\mathcal{Y}_{un})$ represents the set of all permutations of class labels for the unknown samples $\tilde{\mathbf{U}}$. The permutation yielding the maximum value is calculated by the Hungarian optimal assignment algorithm [27].

4) *Implementation Details*: We implemented ORI with PyTorch, Scikit-learn and Faiss. The representation networks, f_θ and f_φ , are based on MLP and autoencoder architectures with layers configured as 784-512-128-32-8 for the USTC-TFC2016 dataset, 83-64-32-16-8 for the CIC-IDS2018 dataset and 30-64-32-16 for the TON-IoT dataset. The ReLU activation function is applied to each hidden layer. Both representation networks are trained for 100 epochs with a batch size of 256, and optimized with the Adam optimizer [26]. For both tasks, the learning rate for f_θ is set to 0.001 for USTC-TFC2016, 0.005 for CIC-IDS2018 and 0.007 for TON-IoT, while for f_φ , the learning rates are $2e-4$, $9e-4$ and $8e-4$, respectively. In open recognition, the number of nearest neighbors k is set to 25, the number of local neighbors k_1 to 50, and the number of reliable samples k_2 to 5. The number of inspected samples M is set to 100 for unknown sample inspection. We used a default temperature parameter τ of 0.07 as provided in [28], and a weight λ of 0.1.

5) *Baselines*: We evaluate the open recognition performance of ORI in the pseudo multi-class classification setting on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets, and in the multi-class classification setting with the first two datasets. Since no prior work has explored open recognition for network intrusion in a multi-class classification setting, we adapt three existing network intrusion open recognition methods (CVAE-EVT [47], OCN [55] and CADE [49]) from the original pseudo multi-class classification setting to a

TABLE II
PERFORMANCE COMPARISON OF ORI WITH OPEN RECOGNITION BASELINES ON THE USTC-TFC2016 AND CIC-IDS2018 DATASETS

Methods	USTC-TFC2016				CIC-IDS2018			
	Known Class Acc.	Open Recognition Acc.	WAF	U-WAF	Known Class Acc.	Open Recognition Acc.	WAF	U-WAF
<i>Pseudo Multi-class Classification Setting</i>								
MSP	0.9975	0.6237	0.6596	0.5289	1.0000	0.7760	0.7260	0.3498
CVAE-EVT	0.9966	0.6396	0.5205	0.7606	1.0000	0.8164	0.8128	0.6521
OCN	0.9927	0.6417	0.6441	0.5538	1.0000	0.9087	0.9062	0.8175
CADE	0.9974	0.8587	0.8592	0.8801	1.0000	0.9338	0.9335	0.8807
ORI	0.9972	0.8695	0.8694	0.8849	1.0000	0.9429	0.9425	0.8967
<i>Multi-class Classification Setting</i>								
MSP	0.9278	0.6869	0.6640	0.7459	0.9998	0.7135	0.6791	0.3970
CVAE-EVT	0.9289	0.6941	0.5908	0.7890	0.9992	0.9118	0.9123	0.8375
OCN	0.9196	0.6345	0.6707	0.6161	0.9920	0.9404	0.9409	0.9030
CADE	0.9256	0.8138	0.8167	0.8476	0.9978	0.9473	0.9474	0.9118
ORI	0.9269	0.8330	0.8223	0.8628	0.9998	0.9794	0.9794	0.9647

multi-class classification setting, serving as baselines for open recognition evaluation. In addition, we use Maximum Softmax Probability (MSP) [18] as an open recognition baseline, which is widely used in non-security applications. **CVAE-EVT** enforces conditional reconstruction constraints and approximates the score distribution for known and unknown classes using extreme value theory modeling to optimize open recognition. **OCN** learns compact representations by jointly optimizing three loss functions and utilizes the nearest class mean classifier with a fixed percentile distance threshold for detecting unknowns. **CADE** extends OCN with explicit distance estimation for known classes, incorporating contrastive learning to compare samples in feature space and detect outliers based on the median absolute deviation distance. **MSP** calculates a confidence score based on the maximum value of the softmax posterior distribution, which is trained with cross-entropy loss in a closed-world environment.

We also evaluate ORI against four unknown sample inspection baselines: two hierarchical clustering methods (**BIRCH** [53] and **FINCH** [39]), one partition-based clustering method (k -means [2]), and one semantic embedding clustering method (**OCN** [55]). **BIRCH** is a memory-efficient method suitable for large datasets that constructs a clustering feature tree to facilitate the clustering process. **FINCH** is a parameter-free method that clusters samples according to the first nearest neighbor of each sample. **OCN**, similar to ORI, first estimates the number of classes with DBSCAN, then clusters the unknown samples using k -means with the estimated number of classes. **K-means** is a popular partition-based method, and we used the ground truth number of classes for comparison.

B. Effectiveness

1) *Open Recognition Performance*: We adapted and re-implemented four open recognition baselines to evaluate the performance of ORI in two settings. Table II presents a comparison of the open recognition results under the pseudo multi-class and multi-class classification settings on the USTC-TFC2016 and CIC-IDS2018 datasets. Table III provides a comparison of the open recognition results in the pseudo multi-class classification settings on the TON-IoT dataset.

TABLE III
PERFORMANCE COMPARISON OF ORI WITH OPEN RECOGNITION BASELINES ON THE TON-IoT DATASET

Methods	TON-IoT			
	Known Class Acc.	Open Recognition Acc.	WAF	U-WAF
MSP	0.9739	0.8188	0.8272	0.6542
CVAE-EVT	0.9730	0.7216	0.6946	0.3170
OCN	0.9362	0.8231	0.8178	0.6396
CADE	0.9663	0.8461	0.8506	0.7206
ORI	0.9829	0.9511	0.9515	0.9170

We can see that MSP demonstrates very low open recognition accuracy and U-WAF scores in both settings, consistently ranking within the bottom two on the USTC-TFC2016 dataset and last on the CIC-IDS2018 dataset. This result can be attributed to the fact that the deep model produces a highly overconfident posterior distribution, even for unknown samples far from the softmax decision boundary. As a result, a large number of unknown samples are misclassified as known classes.

By modeling the reconstruction error of features, CVAE-EVT improves open recognition performance over MSP. The accuracy of open recognition is increased by 0.72% to 1.59% on the USTC-TFC2016 dataset and 4.04% to 19.83% on the CIC-IDS2018 dataset under different settings. Similarly, U-WAF scores show significant improvement, with the smallest gain being 4.31% on the USTC-TFC2016 dataset in a multi-class classification setting. However, this method relies on the assumption that known class samples produce lower reconstruction errors than unknown samples. While this assumption may seem reasonable, it is challenging to reconstruct complex known class samples, such as the USTC-TFC2016 and TON-IoT datasets.

Non-parametric methods OCN, CADE and ORI outperform both MSP and CVAE-EVT, except for OCN in the multi-class classification setting on the USTC-TFC2016 dataset. Although using multiple loss functions to train the representation network, OCN fails to learn effective feature representations for the USTC-TFC2016 dataset. CADE achieves the best open

TABLE IV

PERFORMANCE COMPARISON OF ORI WITH UNKNOWN SAMPLE INSPECTION BASELINES ON THE USTC-TFC2016 AND CIC-IDS2018 DATASETS

Methods	USTC-TFC2016				CIC-IDS2018			
	Unknown		All		Unknown		All	
	Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.
FINCH	47	0.4886	18	0.4298	20	0.7801	51	0.6123
BIRCH	11	0.5250	18	0.4316	29	0.7502	18	0.6352
OCN	25	0.4603	33	0.4317	49	0.6385	43	0.6686
k -means	12	0.6000	20	0.5157	10	0.7309	18	0.6964
ORI $_{k\text{-means}}$	12	0.6497	17	0.5612	10	0.8703	17	0.8307
ORI $_{\text{random}}$	12	0.6507	16	0.5653	10	0.8785	15	0.8377

TABLE V

PERFORMANCE COMPARISON OF ORI WITH UNKNOWN SAMPLE INSPECTION BASELINES ON THE TON-IoT DATASET

Methods	TON-IoT			
	Unknown		All	
	Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.
FINCH	10	0.4334	8	0.5135
BIRCH	2	0.6191	2	0.4325
OCN	19	0.5261	22	0.4842
k -means	4	0.6205	10	0.5467
ORI $_{k\text{-means}}$	4	0.7681	10	0.6546
ORI $_{\text{random}}$	4	0.7458	10	0.6787

recognition performance among the baselines by leveraging contrastive loss and reconstruction loss. In addition, both OCN and CADE use similar strategies to identify unknown samples by measuring the distance between the feature vector of the sample and the centroid of known classes. These experimental results show that the training data distribution affects the recognition performance of these methods, while ORI effectively alleviates this problem.

ORI significantly improves open recognition performance, outperforming the most competitive baseline CADE by 1.08% to 1.92% on the USTC-TFC2016 dataset and by 0.91% to 3.21% on the CIC-IDS2018 dataset under different settings, and by 10.50% on the TON-IoT in the pseudo multi-class classification setting. These results show that ORI achieves superior open recognition performance by combining supervised contrastive learning with non-parametric density estimation, thereby effectively recognizing unknown samples while accurately classifying samples of known classes.

2) *Unknown Sample Inspection Performance:* For a fair comparison in unknown sample inspection, we applied ORI to recognize unknown samples and used the feature vectors extracted by f_ϕ for all baselines. As mentioned earlier, we evaluate two scenarios: one where the samples recognized as unknown include known classes (denoted as “All”) and one without known classes (denoted as “Unknown”).

As shown in TABLE IV and V, the hierarchical clustering methods FINCH and BIRCH accurately estimated the number of classes in most scenarios on the USTC-TFC2016 dataset but struggled with the CIC-IDS2018 and TON-IoT datasets. Among the clustering accuracy, these two methods performed worst in the “All” scenario. It can shed light on the sensitivity of hierarchical clustering methods to noise, such as known class samples misclassified as unknown.

When the number of unknown classes is assumed to be known, k -means demonstrates competitive performance,

TABLE VI

ABLATION COMPARISON OF KEY COMPONENTS IN OPEN RECOGNITION. RESULTS ON THE USTC-TFC2016, CIC-IDS2018 AND TON-IoT DATASETS, WITH $k_1 = 50$ AND $k_2 = 5$

Datasets	Methods	Known Class Acc.	Open Recognition Acc.
USTC-TFC2016	w/o SCL	0.9277	0.4961
	w/o DE	0.9269	0.7866
	ORI	0.9269	0.8330
CIC-IDS2018	w/o SCL	0.8773	0.7433
	w/o DE	0.9998	0.9435
	ORI	0.9998	0.9794
TON-IoT	w/o SCL	0.9659	0.7672
	w/o DE	0.9829	0.8102
	ORI	0.9829	0.9511

second only to ORI. Its clustering accuracy surpasses that of FINCH and BIRCH in the “All” scenarios of three datasets but degrades in the “Unknown” scenarios of the CIC-IDS2018 dataset. These results indicate that while k -means is effective with the ground truth classes, accurately estimating the number of classes for unknown samples remains difficult.

By estimating the number of classes before clustering, OCN exhibited high error rates in the estimated number of classes, particularly in the “All” scenario on three datasets, leading to a significant increase in error relative to BIRCH. Although OCN outperformed FINCH and BIRCH in the “All” scenario for clustering accuracy, it ranked last in the “Unknown” scenario. These results show that over-estimating the number of classes impacts clustering accuracy and increases inspection overhead.

ORI consistently provides superior detection performance on three datasets when using two ways to select representative samples. Both ORI $_{k\text{-means}}$ and ORI $_{\text{random}}$ show comparable performance in estimating the number of classes and clustering accuracy in both scenarios. ORI $_{k\text{-means}}$ excelled in estimating the number of classes, while ORI $_{\text{random}}$ achieved better clustering accuracy. These results indicate that by incorporating reconstruction learning on all samples and constrained clustering, ORI effectively focuses on unknown samples and leverages prior knowledge, ultimately achieving the best performance in estimating the number of classes and clustering accuracy.

C. Ablation

1) *Unknown Sample Recognition:* To evaluate the effectiveness of our open recognition module, we conduct ablation experiments in a multi-class classification setting on the USTC-TFC2016 and CIC-IDS2018 datasets, and in a pseudo

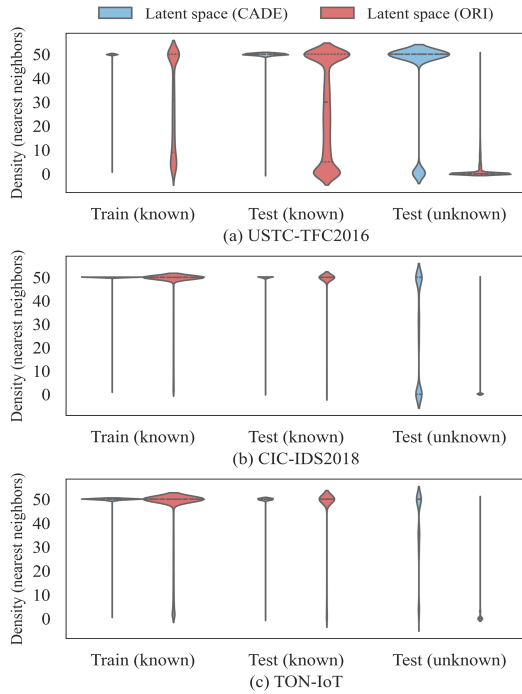


Fig. 3. Violin plots of the nearest neighbor density distributions for known and unknown classes in the latent spaces of CADE and ORI on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets.

multi-class classification setting on the TON-IoT dataset, by comparing the open recognition accuracy before and after removing key components. Specifically, we analyze two main components, including supervised contrastive learning (SCL) for the representation network and density estimation (DE) for recognizing unknown samples. As shown in TABLE VI, both components significantly enhance open recognition performance on three datasets. After ablation, open recognition accuracy drops considerably, while the accuracy for known classes declines only slightly.

Without SCL, the representation network is trained with contrastive learning and reconstruction loss, corresponding to the most competitive baseline CADE. The results show a marked decrease in open recognition accuracy, with a minor reduction in known class accuracy, suggesting that SCL strengthens the ability of the representation network to distinguish between known and unknown classes. Furthermore, SCL plays a critical role in both classifying known classes and recognizing unknown samples.

Without DE, unknown samples are recognized by calculating the distance between feature vectors and class centroids, as in the baseline methods CADE and OCN. In this ablation, we use the 95% percentile as the threshold for each class. As shown in TABLE VI, removing DE results in a substantial reduction in open recognition accuracy across three datasets, with a 4.64% drop on the USTC-TFC2016 dataset, a 3.59% drop on the CIC-IDS2018 dataset and a 14.09% drop on the TON-IoT dataset. The result demonstrates that our proposed density-based open recognition method effectively reduces the open space risk of misclassifying unknown samples as known classes and vice versa.

To provide a more intuitive understanding of how feature representations affect unknown sample recognition by DE, we present a visualization in Fig. 3. The figure displays the violin plots of the nearest neighbor density distribution of known classes in the training data, as well as known and unknown classes in the test samples in CADE and ORI. In the feature space of ORI, the density distribution of training data and test samples for known classes is dense and consistent, while the density distribution of unknown samples is sparse and distinct. In contrast, CADE does not clearly separate known and unknown classes in the USTC-TFC2016 dataset, leading to poor open recognition performance compared to ORI.

2) *Unknown Sample Inspection*: To verify the effectiveness of feature representation and constrained clustering in the unknown sample recognition module, we conducted four ablation experiments on all datasets in the more realistic “All” scenario with $\text{ORI}_{k\text{-means}}$. The comparison results are shown in TABLE VII.

In experiment (a), using the original feature vectors extracted by the feature network f_θ yields the most accurate estimated number of classes but obtains the lowest clustering accuracy for all datasets. Conversely, experiment (b) demonstrates that the accuracy of the estimated number of classes decreases on the USTC-TFC2016 dataset, while the clustering accuracy improves on all datasets. This result indicates that incorporating reconstruction loss enhances the feature representation of unknown samples in the learned representation network.

Experiment (d) introduces joint optimization, we can observe a notable improvement in the performance of unknown sample inspection, surpassing the clustering accuracy achieved when contrastive loss and reconstruction error are used independently. These results suggest that jointly optimizing the feature representation network strengthens the representation of unknown samples by integrating prior knowledge from the training data.

In experiment (c), where clustering constraints are removed, we use k -means to cluster unknown samples based on the previously estimated number of classes. Comparing the results of experiments (c) and (d) reveals that our complete method outperforms in clustering accuracy, with improvements of 3.52% on the USTC-TFC2016 dataset, 4.31% on the CIC-IDS2018 dataset and on 9.44% the TON-IoT dataset. This comparison confirms that constrained clustering is crucial for correcting samples incorrectly clustered by k -means, highlighting the importance of constrained clustering in our method.

D. Sensitivity

1) *The Number of Local Samples k_1 and Reliable Samples k_2* : We evaluate the effect of varying the number of local samples k_1 and reliable samples k_2 on open recognition performance. Recall that k_1 represents the number of nearest neighbor samples retrieved from the training data, while k_2 denotes the number of neighbor samples required to determine if a test sample belongs to a known class. We varied k_1 across values (50, 75, 100, 125, 150, 175, and 200) and selected k_2 from a different range (5, 10, 15, 20, 25, 30, and 50). Fig. 4 shows the open recognition accuracy of our method under these configurations.

TABLE VII

ABLATION COMPARISON OF KEY COMPONENTS IN UNKNOWN SAMPLE INSPECTION. RESULTS ON THE USTC-TFC2016, CIC-IDS2018 AND TON-IoT DATASETS WITH k -MEANS SAMPLING, AND THE NUMBER OF INSPECTED SAMPLES IS 100

Index	Supervised Contrastive Loss	Reconstruction Loss	Constraints	USTC-TFC2016		CIC-IDS2018		TON-IoT	
				Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.	Number of Classes	Cluster Acc.
(a)	✓		✓	18	0.4379	17	0.6187	9	0.4930
(b)		✓	✓	15	0.4581	17	0.7106	10	0.5434
(c)	✓	✓		17	0.5260	17	0.7876	10	0.5602
(d)	✓	✓	✓	17	0.5612	17	0.8307	10	0.6546

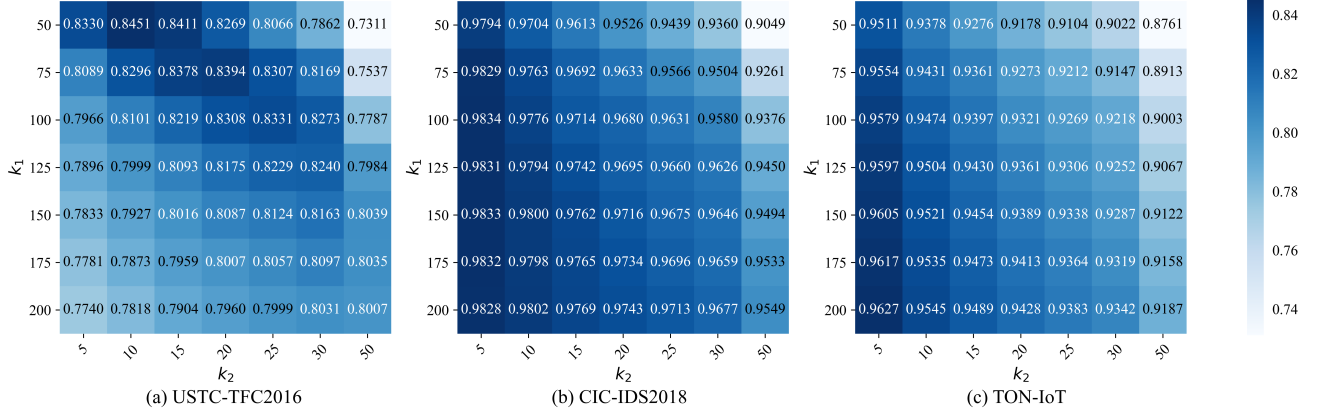


Fig. 4. Performance comparison for different values of k_1 and k_2 on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets with $\alpha = 100\%$.

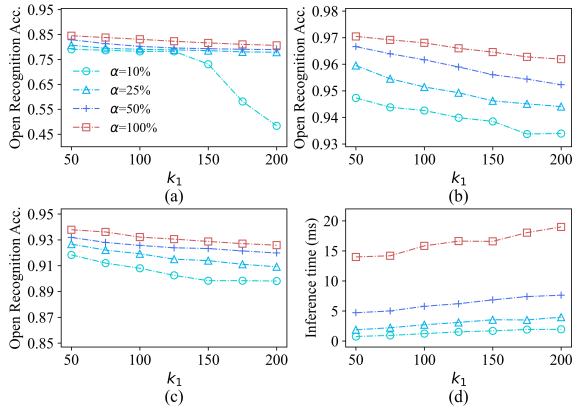


Fig. 5. Comparison with the effects of different k_1 and sampling ratios α . In (a), (b) and (c), we present the open recognition accuracy on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets, and in (d), we compare the inference speed per 100 samples on the CIC-IDS2018 dataset.

In the USTC-TFC2016 dataset, performance improvements are generally observed along the diagonal, indicating that as the number of local samples increases, a corresponding increase in reliable samples is needed for optimal results. In the CIC-IDS2018 dataset, improvements are concentrated in the lower left corner, suggesting that a higher number of local samples reduces the need for reliable samples. This difference is because the CIC-IDS2018 dataset has $5.4 \times$ more training data than the USTC-TFC2016 dataset. The results also indicate that ORI exhibits excellent stability and enhanced open recognition performance when sufficient training data is available.

2) *Sampling Ratio α* : We also examine the impact of varying random sampling ratios α on open recognition

performance. We set four sampling ratio values (10%, 25%, 50%, and 100%), with a fixed 5:1 ratio between the number of local samples k_1 and reliable samples k_2 . Fig. 5 (a)-(c) shows the open recognition accuracy results across these sampling ratios. The open recognition accuracy remains relatively stable across varying random sampling ratios when k_1 is small on the USTC-TFC2016 dataset. In the CIC-IDS2018 and TON-IoT datasets, open recognition accuracy decreases as the sampling ratio decreases and k_1 increases. Fig. 5(d) further illustrates the effect of different sampling ratios and k_1 values on inference speed. Notably, k_1 changes do not significantly affect inference speed at 10% and 25% sampling ratios. However, the inference speed decreases as k_1 increases when using the entire training data.

3) *The Number of Inspected Samples M* : We evaluate how varying the number of inspected samples M influences unknown sample inspection. We considered M values of (50, 100, 150, 200, 250, 300, and 500). Fig. 6 presents the clustering accuracy and estimated number of class results. As shown in Fig. 6 (a), (c) and (e), clustering accuracy improves significantly on three datasets with a few inspected samples. When the number of inspected samples exceeds 100, the results become more variable. In the USTC-TFC2016 dataset, an increase in the number of inspected samples leads to a decline in clustering performance, whereas in the CIC-IDS2018 and TON-IoT datasets, accuracy continues to improve but eventually stabilizes as the number of inspected samples grows. The results indicate that ORI benefits from a limited number of inspection samples for labeling unknown samples. However, the clustering accuracy did not improve when the number of samples exceeded a certain threshold.

It can be seen in Fig. 6 (b), (d) and (f) that as the number of inspected samples increases, the estimated

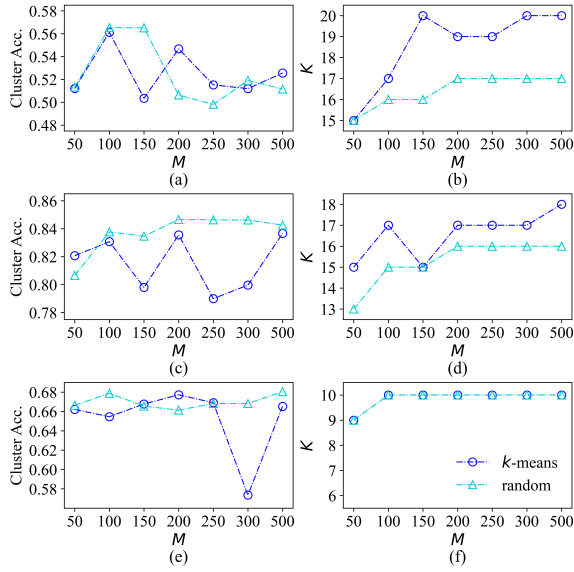


Fig. 6. Comparison of the effect of different M values and sampling strategies. We report the clustering accuracy and estimated number of classes for the USTC-TFC2016 dataset in (a) and (b), the CIC-IDS2018 dataset in (c) and (d), and the TON-IoT dataset in (e) and (f), respectively.

number of unknown classes K gradually approaches the ground truth value. For instance, $\text{ORI}_{k\text{-means}}$ accurately estimates the number of unknown classes when 100 inspected samples in the USTC-TFC2016 dataset. In $\text{ORI}_{\text{random}}$, the estimated number of classes stabilizes once the number of inspected samples exceeds 200.

E. Real-World Inspection

To effectively utilize labeled samples, it is essential to evaluate the labels of unknown samples to estimate the noise ratio in the labeled data or to maximize the selection of correctly labeled samples. Given the labeled samples in the “All” scenario, we simulate a security analyst to inspect the labeling results. Since the labeling results may not perfectly match the ground truth labels, it is essential to rank the samples to prioritize those with high-confidence labels. Specifically, we sort the samples by their distances from the cluster centers and divide them into equal-sized intervals. For each interval, a subset of samples from each interval is then inspected by security analysts to estimate the labeling accuracy of the interval. We also introduce an unknown sample inspection ratio to quantify the effort needed to ensure that all sample labels are entirely accurate. The effort for each interval is defined as the total number of samples in the interval when its clustering accuracy is below 100%.

We conduct an experimental analysis on three datasets using $\text{ORI}_{k\text{-means}}$. The samples are divided into 100 intervals, with ten samples selected from each interval for inspection. Fig. 7 shows the ground truth clustering accuracy, the estimated clustering accuracy for each interval, and the ratio of accurately labeled samples. In the USTC-TFC2016 dataset, the top 40% of intervals achieved an accuracy above 60%, while in the CIC-IDS2018 dataset, up to 80% of intervals exhibited high labeling accuracy. Although the estimated and actual

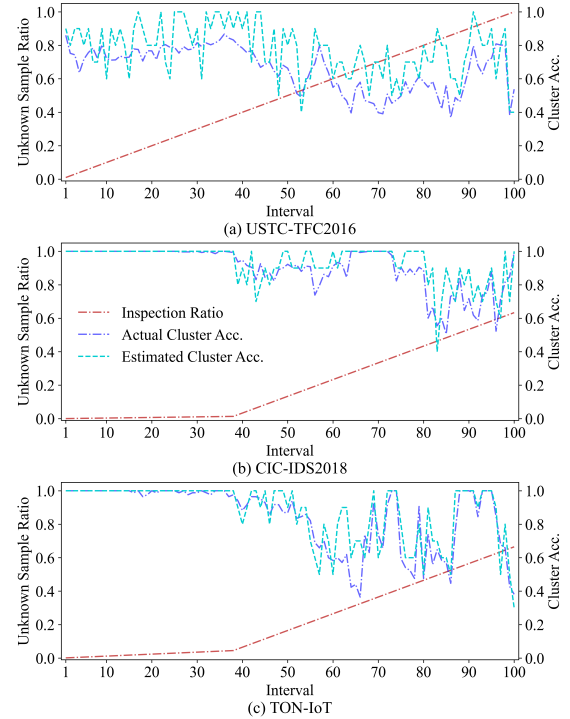


Fig. 7. Comparison of actual cluster accuracy, estimated cluster accuracy, and inspection efforts on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets, with unknown samples ranked by their distance to the corresponding centroid.

accuracies differ slightly, the estimated result is still valuable for identifying high-confidence labeled samples. For instance, in the USTC-TFC2016 dataset, the accuracy of the top 40% of intervals ranged from 60% to 100%, and in the CIC-IDS2018 dataset, the accuracy of the top 80% of intervals consistently exceeded 70%. In the TON-IoT dataset, the accuracy of more than the top 50% of intervals consistently exceeded 80%.

By aggregating the estimated accuracy for each interval, we simulate the effort required to label all samples accurately. The results show that the ratio of inspected unknown samples increased as the interval accuracy dropped below 100% on three datasets. Notably, in the CIC-IDS2018 dataset, the estimated accuracy for the first 38 intervals is 100%, matching the ground truth accuracy, which dramatically reduces the effort required by security analysts and allows them to focus on more challenging samples. The analysis demonstrates that the inspection results of ORI for unknown samples are effective, enabling security analysts to quickly identify high-quality labeled samples and significantly reduce manual inspection overhead.

V. RELATED WORK

A. Network Intrusion Detection

Network intrusion detection is dedicated to identifying attacks in network traffic. Traditional methods primarily rely on generating specific detection rules for known attack patterns, which are then applied to intrusion detection systems, such as Snort or Suricata, to discover malicious activities in real time [3], [30], [42], [52]. While these rules can be highly accurate for identifying known attacks, they often face

challenges when detecting unknown attacks or variants of known attacks designed to evade detection.

Deep learning-based network intrusion detection has been explored to tackle unknown attacks, involving two stages: feature engineering and model building. Feature engineering refers to extracting features from network traffic, such as extracting statistical features from flows [38] or transforming flows into images [44], [48]. Although different feature engineering methods significantly impact the performance of deep models, this work does not focus on comparing existing methods or developing new ones. There are various types of deep models built to meet different security needs, which can be broadly categorized into anomaly detection models, binary classification models, pseudo multi-class classification models, and multi-class classification models. Mirsky et al. [32] proposed Kitsune, which utilizes an ensemble auto-encoder network to distinguish between normal and abnormal traffic patterns through anomaly scores. Han et al. [22] introduced an online evolutionary anomaly detection framework that mitigates noise and dynamically updates model parameters. Although these anomaly detection models can identify unknown attacks, they often lead to a high rate of false positives. Ding et al. [7] used a generative adversarial network to create diverse attack data, enhancing classification accuracy. Hu et al. [23] proposed an early and accurate detection method using flow graphs and graph2vec for classification, while Zhang et al. [54] proposed Metis, which employs byte-level tagging and semi-supervised knowledge distillation to enhance network rule processing. However, these methods are designed for static environments and cannot recognize and inspect unknown samples.

Our work focuses on open-world network intrusion recognition and inspection. To our knowledge, most existing work primarily focuses on open recognition while paying little attention to the inspection of unknown samples. The OCN [55] used a distance-based method for open set recognition to identify unknown attacks and then cluster unknown samples by estimating the number of classes. However, due to limitations in effectively modeling known class data and significant errors in estimating the number of classes, OCN performs significantly worse than ORI in both open recognition and unknown sample detection.

B. Open Recognition

Recent advances in open-world network intrusion detection include open set recognition and concept drift detection. Han et al. [19] introduced OWAD, an open-world anomaly detection framework that employs permutation tests to detect drifted anomalous samples by comparing distributional differences between training data and incoming data. Ping and Ye [34] proposed OpenIDS for open set network intrusion detection in a binary classification setting, which uses a minimum-maximum autoencoder to distinguish between known and unknown samples and integrates a pseudo-extreme value machine. However, these methods often fail to provide detailed attack information, which hinders timely responses in practical scenarios. CVAE-EVT [47] formalized fine-grained open set network intrusion detection as a two-stage problem, optimizing two scores to minimize risks associated with misclassifying

known attacks and misrecognizing unknown attacks. OCN [55] developed an open set classifier using convolutional neural networks for feature representation, which further employs a class means classifier for classifying known class samples and rejecting unknown ones. CADE [49] extended feature representation learning by incorporating reconstruction and contrastive losses, using the mean absolute distance to identify samples that deviate from training data.

Most studies have focused on pseudo multi-class classification, and no existing methods tackle multi-class classification open recognition in network intrusion detection. While CVAE-EVT groups normal samples into fine-grained classes through clustering, our work explores using fine-grained data with ground truth labels. The comparison results indicate that existing open recognition methods are inferior to ORI, primarily due to their lack of adaptability to the underlying training distribution, which increases the risk of open space.

3) *Unknown Sample Inspection*. The inspection of unknown samples is still an emerging area. Most existing methods assume that security analysts can individually inspect and label a few samples recognized as unknown, which is then used to enhance the performance of deep models [19], [49]. However, this assumption is impractical since labeling is labor-intensive and time-consuming, and unknown samples continuously emerge. In contrast, we aim to inspect and label all unknown samples with minimal overhead. Zhang et al. [55] proposed clustering in semantic space using metric distances of unknown samples, initially employing DBSCAN to estimate the number of classes, followed by k -means clustering. However, this method faces challenges in accurately estimating class numbers, which can affect clustering accuracy. In comparison, ORI simplifies the process by selecting a few unknown samples for inspection, subsequently using these inspected samples as constraints to assign labels for the remaining unknown samples via clustering. We hope our work draws strong attention to the importance of unknown sample inspection in network security.

VI. DISCUSSION

A. Feature Extraction and Open Recognition

As discussed in Section IV-B, the selection of feature extraction methods significantly influences the performance of network intrusion detection models in open-world environments. Although prior studies have extensively assessed these methods in closed-world assumptions, their effectiveness in open-world environments remains largely unexplored. Hence, it is crucial to comprehensively evaluate various feature extraction methods to develop more robust open-world network intrusion detection models. In addition, given the great potential of large language models (LLMs) in network security, it is also worth paying attention to developing open-world network intrusion detection models with LLMs.

B. Incremental Updates

The incremental updating of deep models is critical for achieving continuous detection of network intrusions in open-world environments. As shown in Section IV-E, by simulating a security analyst inspecting labeled unknown samples, we

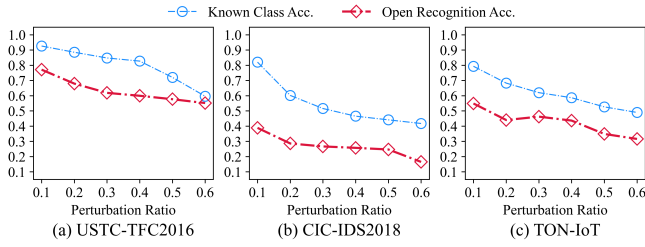


Fig. 8. Known class and open recognition accuracies with various perturbation ratios on the USTC-TFC2016, CIC-IDS2018 and TON-IoT datasets.

can obtain samples with many noisy labels and only a few correctly labeled samples. Since the incremental update of deep models is beyond the scope of this work, we plan to supplement our system in the future by retraining deep models using samples with noisy labels, incrementally updating deep models with a few correctly labeled samples, and adapting to the distribution changes of normal data [19]. In addition, as network traffic distributions change over time, we will explore integrating a drift detection module with incremental learning or streaming clustering methods to dynamically update the representation network or clustering results, thereby maintaining sensitivity to emerging attack patterns.

C. Adversarial Attacks

Adversarial samples pose a critical challenge to machine learning-based network intrusion detection systems, as they can manipulate the predictions of well-trained models by injecting deliberately crafted perturbations into network traffic. These perturbations are typically generated using gradient-based methods. However, such techniques do not apply to ORI, which predominantly relies on the density distribution of samples within the feature space. To assess ORI's resilience against adversarial samples, we generated adversarial samples that lie between known and unknown classes. For example, to generate the adversarial sample x_{adv} from a known sample x , we first identify the reference sample x_{ref} with the shortest latent distance to x in the unknown sample dataset $\mathcal{D}_{un}$. Then, we perturb x incrementally by replacing its feature values with the corresponding feature values of x_{ref} , making the smallest possible change while ensuring that the perturbations remain meaningful. To quantify the perturbation, we use the ratio of the distance between x_{adv} and x as a measure of the perturbation. We set the range of perturbation size as 10%, 20%, 30%, 40%, 50% and 60%. As shown in Fig. 8, it can be seen that both known class accuracy and open recognition accuracy decrease as the perturbation ratio increases, particularly in the CIC-IDS2018 and TON-IoT datasets. We conclude that ORI is vulnerable to adversarial examples, and defense mechanisms against such attacks need to be explored in future work.

D. Limitations and Future Work

Our work has some limitations. First, the hyperparameters of ORI for recognizing unknown samples are determined experimentally, including the number of local samples k_1 and the number of reliable samples k_2 . Given the hyperparameter sensitivity analysis in Section IV-D, we will explore automatic tuning via a held-out validation set or develop a

dynamic threshold adjustment strategy to enable the system to adaptively select optimal parameters for different traffic distributions in future work. Second, we propose that unknown sample inspection is a one-shot process, and labeling accuracy does not improve with the increasing number of inspected samples. However, inspecting unknown samples may be an iterative retrieval process in practice. A more effective strategy is to use active learning techniques [29] to enhance the inspection of unknown samples.

VII. CONCLUSION

In this paper, we develop a novel network intrusion detection system, ORI, to classify known class samples, recognize unknown samples and inspect samples recognized as unknown. ORI introduces several techniques: 1) a novel density-based method for unknown sample recognition guarantees that ORI can accurately identify unknown samples based on the known data distribution. 2) a constrained clustering method to help security analysts label all unknown samples with limited manual inspection overhead. Experimental results show that ORI outperforms existing baseline methods in open recognition and unknown sample inspection. We also analyze the unknown sample labeling results provided by ORI to support future model updates.

REFERENCES

- [1] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1851–1877, 2nd Quart., 2019, doi: [10.1109/COMST.2019.2891891](https://doi.org/10.1109/COMST.2019.2891891).
- [2] D. Arthur and S. Vassilvitskii, "K-means++: The advantages of careful seeding," in *Proc. ACM-SIAM Symp. Discrete Algorithms*, Apr. 2007, pp. 1027–1035.
- [3] L. Alcantara, G. Padilha, R. Abreu, and M. d'Amorim, "Syrius: Synthesis of rules for intrusion detectors," *IEEE Trans. Rel.*, vol. 71, no. 1, pp. 370–381, Mar. 2022, doi: [10.1109/TR.2021.3061297](https://doi.org/10.1109/TR.2021.3061297).
- [4] A. Bendale and T. Boulton, "Towards open world recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 1893–1902.
- [5] S. Cruz, C. Coleman, E. M. Rudd, and T. E. Boulton, "Open set intrusion recognition for fine-grained attack categorization," in *Proc. IEEE Int. Symp. Technol. Homeland Secur. (HST)*, Apr. 2017, pp. 1–6, doi: [10.1109/THS.2017.7943467](https://doi.org/10.1109/THS.2017.7943467).
- [6] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4766, May 2023, doi: [10.1109/TKDE.2022.3142820](https://doi.org/10.1109/TKDE.2022.3142820).
- [7] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1156–1167, 2024, doi: [10.1109/TIFS.2023.3331240](https://doi.org/10.1109/TIFS.2023.3331240).
- [8] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024, doi: [10.1109/TNSM.2023.3332284](https://doi.org/10.1109/TNSM.2023.3332284).
- [9] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [10] T. E. T. Djaidja, B. Brik, S. M. Senouci, A. Boualouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and RNNs," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7783–7793, 2024, doi: [10.1109/TIFS.2024.3441862](https://doi.org/10.1109/TIFS.2024.3441862).
- [11] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7–12, doi: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009).
- [12] C. Geng, S.-J. Huang, and S. Chen, "Recent advances in open set recognition: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3614–3631, Oct. 2021, doi: [10.1109/TPAMI.2020.2981604](https://doi.org/10.1109/TPAMI.2020.2981604).

- [13] C. Gao, G. Huang, H. Li, B. Wu, Y. Wu, and W. Yuan, "A comprehensive study of learning-based Android malware detectors under challenging environments," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Lisbon, Portugal, Feb. 2024, pp. 1–13, doi: [10.1145/3597503.3623320](https://doi.org/10.1145/3597503.3623320).
- [14] H. Gao, S. Cheng, and W. Zhang, "GDroid: Android malware detection and classification with graph convolutional network," *Comput. Secur.*, vol. 106, Jul. 2021, Art. no. 102264.
- [15] J. L. Guerra, C. Catania, and E. Veas, "Datasets are not enough: Challenges in labeling network traffic," *Comput. Secur.*, vol. 120, pp. 1–17, Sep. 2022.
- [16] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, Apr. 2014.
- [17] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, Feb. 2009, doi: [10.1016/j.cose.2008.08.003](https://doi.org/10.1016/j.cose.2008.08.003).
- [18] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. ICLR*, 2017, pp. 1–11.
- [19] D. Han et al., "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [20] J. Halvorsen, C. Izurieta, H. Cai, and A. Gebremedhin, "Applying generative machine learning to intrusion detection: A systematic mapping study and review," *ACM Comput. Surv.*, vol. 56, no. 10, pp. 1–33, Oct. 2024, doi: [10.1145/3659575](https://doi.org/10.1145/3659575).
- [21] J. Henrydoss, S. Cruz, E. M. Rudd, M. Gunther, and T. E. Boulton, "Incremental open set intrusion recognition using extreme value machine," in *Proc. 16th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Aug. 2017, pp. 1089–1093, doi: [10.1109/ICMLA.2017.000-3](https://doi.org/10.1109/ICMLA.2017.000-3).
- [22] S. Han et al., "Log-based anomaly detection with robust feature extraction and online learning," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2300–2311, 2021, doi: [10.1109/TIFS.2021.3053371](https://doi.org/10.1109/TIFS.2021.3053371).
- [23] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023, doi: [10.1109/TIFS.2023.3318960](https://doi.org/10.1109/TIFS.2023.3318960).
- [24] J. Jeon, B. Jeong, S. Baek, and Y.-S. Jeong, "Static multi feature-based malware detection using multi SPP-net in smart IoT environments," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2487–2500, 2024, doi: [10.1109/TIFS.2024.3350379](https://doi.org/10.1109/TIFS.2024.3350379).
- [25] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "MAGIC: Detecting advanced persistent threats via masked graph representation learning," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Apr. 2024, pp. 5197–5214.
- [26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2015, pp. 1–15.
- [27] H. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logistics Quart.*, vol. 2, no. 1, pp. 83–97, 1955.
- [28] P. Khosla et al., "Supervised contrastive learning," in *Proc. NIPS*, 2020, pp. 18661–18673.
- [29] Y. Kim, G. Dán, and Q. Zhu, "Human-in-the-loop cyber intrusion detection using active learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8658–8672, 2024, doi: [10.1109/TIFS.2024.3434647](https://doi.org/10.1109/TIFS.2024.3434647).
- [30] S. Lee et al., "LARGen: Automatic signature generation for malwares using latent Dirichlet allocation," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 5, pp. 771–783, Sep. 2018, doi: [10.1109/TDSC.2016.2609907](https://doi.org/10.1109/TDSC.2016.2609907).
- [31] T. Lu and J. Wang, "DOMR: Toward deep open-world malware recognition," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1455–1468, 2024, doi: [10.1109/TIFS.2023.3338469](https://doi.org/10.1109/TIFS.2023.3338469).
- [32] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2018, pp. 1–15.
- [33] T. M. Booi, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T. H. Den Hartog, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 485–496, Jan. 2022, doi: [10.1109/JIOT.2021.3085194](https://doi.org/10.1109/JIOT.2021.3085194).
- [34] G. Ping and X. Ye, "Open-set intrusion detection with Min-Max autoencoder and pseudo extreme value machine," in *Proc. Int. Jt. Conf. Neural Netw. (IJCNN)*, 2022, pp. 1–8, doi: [10.1109/IJCNN55064.2022.9892858](https://doi.org/10.1109/IJCNN55064.2022.9892858).
- [35] J. Qiu, J. Zhang, W. Luo, L. Pan, S. Nepal, and Y. Xiang, "A survey of Android malware detection with deep neural models," *ACM Comput. Surv.*, vol. 53, no. 6, pp. 1–36, Nov. 2021.
- [36] E. M. Rudd, A. Rozsa, M. Günther, and T. E. Boulton, "A survey of stealth malware attacks, mitigation measures, and steps toward autonomous open world solutions," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 1145–1172, 2nd Quart., 2017, doi: [10.1109/COMST.2016.2636078](https://doi.org/10.1109/COMST.2016.2636078).
- [37] H. Studiawan, F. Sohel, and C. Payne, "Anomaly detection in operating system logs with deep learning-based sentiment analysis," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2136–2148, Sep. 2021, doi: [10.1109/TDSC.2020.3037903](https://doi.org/10.1109/TDSC.2020.3037903).
- [38] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, Mar. 2018, pp. 108–116.
- [39] S. Sarfraz, V. Sharma, and R. Stiefelhagen, "Efficient parameter-free clustering using first neighbor relations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8934–8943.
- [40] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, Oakland, CA, USA, May 2010, pp. 305–316, doi: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25).
- [41] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boulton, "Toward open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 7, pp. 1757–1772, Jul. 2013, doi: [10.1109/TPAMI.2012.256](https://doi.org/10.1109/TPAMI.2012.256).
- [42] R. Uetz, M. Herzog, L. Hackländer, S. Schwarz, and M. Henze, "You cannot escape me: Detecting evasions of SIEM rules in enterprise networks," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Apr. 2024, pp. 5179–5196.
- [43] N. Wang, Y. Chen, Y. Xiao, Y. Hu, W. Lou, and Y. T. Hou, "MANDA: On adversarial example detection for network intrusion detection system," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1139–1153, Mar. 2023, doi: [10.1109/TDSC.2022.3148990](https://doi.org/10.1109/TDSC.2022.3148990).
- [44] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717, doi: [10.1109/ICOIN.2017.7899588](https://doi.org/10.1109/ICOIN.2017.7899588).
- [45] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020, doi: [10.1109/TIFS.2020.2991876](https://doi.org/10.1109/TIFS.2020.2991876).
- [46] J. Yang, Q. Zhang, X. Jiang, S. Chen, and F. Yang, "Poirot: Causal correlation aided semantic analysis for advanced persistent threat detection," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3546–3563, Sep. 2022, doi: [10.1109/TDSC.2021.3101649](https://doi.org/10.1109/TDSC.2021.3101649).
- [47] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021, doi: [10.1109/TIFS.2021.3083422](https://doi.org/10.1109/TIFS.2021.3083422).
- [48] J. Yang, H. Li, S. Shao, F. Zou, and Y. Wu, "FS-IDS: A framework for intrusion detection based on few-shot learning," *Comput. Secur.*, vol. 122, Nov. 2022, Art. no. 102899, doi: [10.1016/j.cose.2022.102899](https://doi.org/10.1016/j.cose.2022.102899).
- [49] L. Yang et al., "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. USENIX Secur.*, Mar. 2021, pp. 2327–2344.
- [50] C. Zhang, D. Jia, L. Wang, W. Wang, F. Liu, and A. Yang, "Comparative research on network intrusion detection methods based on machine learning," *Comput. Secur.*, vol. 121, Oct. 2022, Art. no. 102861, doi: [10.1016/j.cose.2022.102861](https://doi.org/10.1016/j.cose.2022.102861).
- [51] N. Zhao et al., "An empirical investigation of practical log anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA, Aug. 2021, pp. 1404–1415, doi: [10.1145/3468264.3473933](https://doi.org/10.1145/3468264.3473933).
- [52] R. Zhang, M. Tong, L. Chen, J. Xue, W. Liu, and F. Xie, "CMIRGen: Automatic signature generation algorithm for malicious network traffic," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Mali, Dec. 2020, pp. 736–743, doi: [10.1109/TRUSTCOM50675.2020.00101](https://doi.org/10.1109/TRUSTCOM50675.2020.00101).
- [53] T. Zhang, R. Ramakrishnan, and M. Livny, "BIRCH: An efficient data clustering method for very large databases," *ACM Sigmod Record*, vol. 25, no. 2, pp. 103–114, doi: [10.1145/235968.233324](https://doi.org/10.1145/235968.233324).
- [54] Z. Zhang et al., "Metis: Understanding and enhancing in-network regular expressions," in *Proc. Adv. Neural Inf. Processing Syst.*, vol. 36, 2023, pp. 77867–77881.

- [55] Z. Zhang, Y. Zhang, D. Guo, and M. Song, "A scalable network intrusion detection system towards detecting, discovering, and learning unknown attacks," *Int. J. Mach. Learn. Cybern.*, vol. 12, no. 6, pp. 1649–1665, Jun. 2021, doi: [10.1007/s13042-020-01264-7](https://doi.org/10.1007/s13042-020-01264-7).



Lei Du received the master's degree from Hebei University of Science and Technology in 2021. He is currently with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. He is also with the Pengcheng Laboratory, Shenzhen. His research focuses on cyberspace security, machine intelligence, and open-world learning.



Yuhua Chai received the Ph.D. degree from the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China, in 2023. She is currently a Post-Doctoral Researcher at the Cyberspace Institute of Advanced Technology, Guangzhou University. Her research interests include cybersecurity, malware detection, and software supply chain security.



Yan Jia is a Professor with the College of Computer Science, National University of Defense Technology, Changsha, China. As a Principal Investigator, she has undertaken more than 20 national projects, including the National Key Project of the 863 Program and the National Natural Science Foundation of China. Her research interests cover big data analysis, artificial intelligence, online social network analysis, and security situation awareness and analysis in cyberspace.

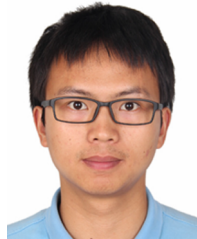


work security, and artificial intelligence security. He is also a member of Chinese Academy of Engineering, Beijing.

Binxiang Fang (Member, IEEE) received the M.S. degree in computer science and technology from Tsinghua University, Beijing, China, in 1984, and the Ph.D. degree in computer science and technology from Harbin Institute of Technology, Harbin, China, in 1989. He is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China, and also with the Department of New Networks, Pengcheng Laboratory, Shenzhen. His current research interests include computer networks, information and network security, and artificial intelligence security. He is also a member of Chinese Academy of Engineering, Beijing.



Hao Li received the M.S. and Ph.D. degrees in electronics and communication engineering from the Communication University of China in 2015 and 2018, respectively. He was a Senior Engineer at the National Key Laboratory of Advanced Communication Networks before 2025 and is currently an Artificial Intelligence Expert at Kunlun Digital Technology Company Ltd. His research interests include artificial intelligence and its security, digital rights management, cloud computing security, and attribute-based encryption.



cyberspace security, cyber range, big data analysis, and artificial intelligence security.

Zhaoquan Gu (Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Tsinghua University in 2011 and 2015, respectively. He was a Professor and the Associate Dean of the Cyberspace Institute of Advanced Technology (CIAT), Guangzhou University, China. He is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. He is also a Professor with the Department of New Networks, Pengcheng Laboratory, Shenzhen. His research interests include